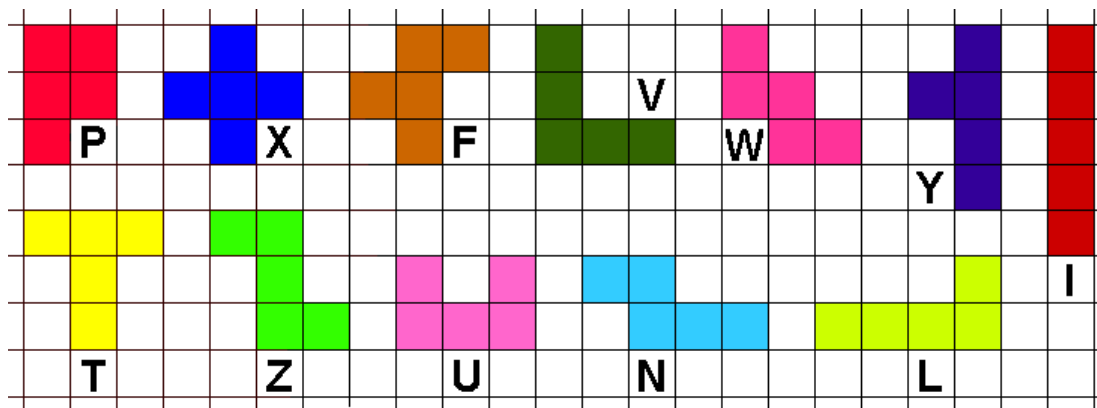


Prática Puzzle – Pentaminós com grafos, buscas e AVL

Os pentaminós são figuras geométricas compostas por cinco quadrados conectados. Existem 12 peças distintas e um dos problemas clássicos consiste em organizá-las em um tabuleiro sem sobreposição.



Este trabalho tem como objetivo desenvolver um sistema computacional que permita resolver e/ou jogar o problema dos pentaminós, utilizando conceitos fundamentais de:

- Grafos
- Busca em Largura (BFS)
- Busca em Profundidade (DFS)
- Árvores balanceadas (AVL)

Objetivo Geral

Desenvolver uma aplicação que permita:

1. Definir dimensões arbitrárias de um tabuleiro (ex: 5×6, 6×10, etc.)
2. Escolher entre:
 - Modo Jogar (interativo)
 - Modo Resolver (automático)
3. Utilizar algoritmos de busca em grafos para encontrar soluções
4. Utilizar uma árvore AVL para controle de estados visitados

Requisitos Funcionais

1. Entrada de Dados

O sistema deve permitir ao usuário:

- Definir dimensões do tabuleiro:
 - Linhas (m)
 - Colunas (n)
- Validar se a área é compatível com o problema:
 - Área ≥ 5
 - Idealmente múltipla de 5

2. Representação do Problema

O sistema deve modelar:

- O tabuleiro como uma matriz bidimensional
- As peças de pentaminós com suas variações (rotações e reflexões)
- Um estado como:
 - Configuração atual do tabuleiro
 - Conjunto de peças já utilizadas

3. Modelagem com Grafos

O problema deve ser representado como um grafo onde:

- Cada nó representa um estado do tabuleiro
- Cada aresta representa a inserção válida de uma peça

4. Modo Resolver (Automático)

O sistema deve oferecer:

- Busca em Profundidade (DFS)
 - Encontrar:
 - Uma solução
 - Ou todas as soluções possíveis
- Busca em Largura (BFS)
 - Encontrar a solução com menor profundidade
- Comparação
 - Tempo de execução
 - Número de estados explorados

5. Uso de Árvore AVL

A aplicação deve implementar uma árvore AVL para:

- Armazenar estados já visitados
- Evitar exploração redundante

Requisitos:

- Cada estado deve ser convertido em uma chave única
- Operações:
 - Inserção
 - Busca

6. Modo Jogar (Interativo)

O sistema deve permitir ao usuário:

- Inserir peças manualmente no tabuleiro
- Visualizar o estado atual
- Validar jogadas:
 - Sem sobreposição
 - Dentro dos limites
- Receber feedback:
 - Jogada válida/inválida

Opcional:

- Sugestão de jogadas possíveis

7. Saída de Dados

O sistema deve:

- Exibir soluções encontradas (texto ou visual)
- Indicar:
 - Número de soluções
 - Tempo de execução
- Permitir visualização passo a passo (opcional)

Requisitos Não Funcionais

- Código modular e organizado
- Documentação clara
- Execução em tempo razoável para entradas pequenas/médias
- Linguagem livre (Java, Python, C++, etc.)

Requisitos Acadêmicos

O relatório deve conter:

1. Modelagem

- Como o problema foi representado como grafo

2. Algoritmos

- Implementação de BFS e DFS
- Diferenças observadas

3. Estrutura AVL

- Justificativa do uso
- Complexidade das operações

4. Análise de Desempenho

- Tempo de execução
- Número de estados visitados
- Comparação entre abordagens

5. Dificuldades e Soluções

6. Relatório de como foi a utilização de IA, caso seja o caso.

Critérios de Avaliação

Critério	Peso
Correção funcional	30%
Uso adequado de grafos	15%
Implementação de BFS e DFS	15%
Implementação da AVL	15%
Qualidade do código	10%
Relatório técnico	15%

Extensões (Bônus)

- Interface gráfica
- Heurísticas (ex: escolha de peça mais restritiva)
- Paralelização
- Visualização animada da busca

Observações

- O problema possui crescimento exponencial
- Recomenda-se testar inicialmente com:
 - Tabuleiros pequenos
 - Subconjuntos de peças

Entrega

- Código-fonte
- Executável (ou instruções de execução)
- Relatório técnico (PDF)
- Demonstração (opcional, mas eu tenho que conseguir jogar)

Dica Final

Organize seu sistema separando claramente:

- Representação do estado
- Geração de vizinhos
- Estratégia de busca
- Estrutura de dados (AVL)

Boa implementação!

